

Database Indexing and Query Optimization

What is a Database Index

A database index is a data structure that improves the speed of data retrieval operations on a database table. An index is created on one or more columns of a table. Without an index, the database must scan every row in the table to find matching records (a full table scan). With an index, the database can quickly locate the relevant rows. The most common index type is the B-tree index, which maintains sorted data and allows searches, insertions, and deletions in logarithmic time.

B-Tree Index Structure

A B-tree is a self-balancing tree data structure that maintains sorted data. Each node in a B-tree can contain multiple keys and multiple child pointers. This makes B-trees shallower than binary trees for the same number of keys, which means fewer disk reads during lookups. The database reads data in fixed-size blocks called pages, and B-trees are designed so that each node fits within a single page. The root node is always kept in memory, so a typical lookup requires only a few disk reads even for very large tables.

Composite Indexes

A composite index is an index on multiple columns. The order of columns in a composite index matters significantly. The index is sorted by the first column, then by the second column within groups of the same first column value, and so on. For example, an index on (last_name, first_name) can efficiently serve queries that filter by last_name alone, or by both last_name and first_name. However, it cannot efficiently serve queries that filter by first_name alone. This is known as the leftmost prefix rule.

Index Selectivity

Selectivity is a measure of how many rows an index will filter out. A highly selective index filters out most rows, leaving a small result set. For example, a unique index has perfect selectivity because each value matches at most one row. The query optimizer uses selectivity to decide whether to use an index. If an index is not selective enough (for example, an index on a gender column with only two values), the optimizer may prefer a full table scan because the overhead of reading the index and then fetching the rows is higher than just scanning the table.

Covering Indexes

A covering index is an index that contains all the columns needed by a query. When a covering index is available, the database can satisfy the query entirely from the index without accessing the table data. This is called an index-only scan. Index-only scans are significantly faster than regular index lookups because they avoid the random I/O of fetching table rows. For example, if a query selects `first_name` and `last_name` from a `users` table, an index on `(last_name, first_name)` can serve as a covering index.

Query Execution Plans

A query execution plan describes how the database will execute a SQL statement. It includes information about which tables are accessed, which indexes are used, the join methods employed, and the estimated cost of each operation. The query optimizer generates the plan by evaluating multiple possible execution strategies and choosing the one with the lowest estimated cost. Developers can use the `EXPLAIN` command to view the execution plan and identify performance issues such as missing indexes or inefficient join strategies.

Index Maintenance and Trade-offs

Indexes improve read performance but slow down write operations. Each `INSERT`, `UPDATE`, or `DELETE` must update all affected indexes. Therefore, adding too many indexes can degrade write performance and increase storage usage. Indexes also need maintenance: over time, indexes can become fragmented due to page splits and deletions. Fragmented indexes have lower cache hit rates and more disk I/O. Regular index rebuilding or reorganizing can restore performance. The decision to add an index should consider the ratio of reads to writes for the affected table, the query patterns, and the available storage.

Full-Text Search Indexes

Standard B-tree indexes are not suitable for text search because they match exact values or prefixes. Full-text search uses a specialized index structure called an inverted index. An inverted index maps each word (token) to a list of documents that contain it. The text is first processed through tokenization (splitting into words), stemming (reducing to root forms), and stop word removal (removing common words like 'the' and 'is'). This allows efficient searching for keywords within large text fields.